universität freiburg

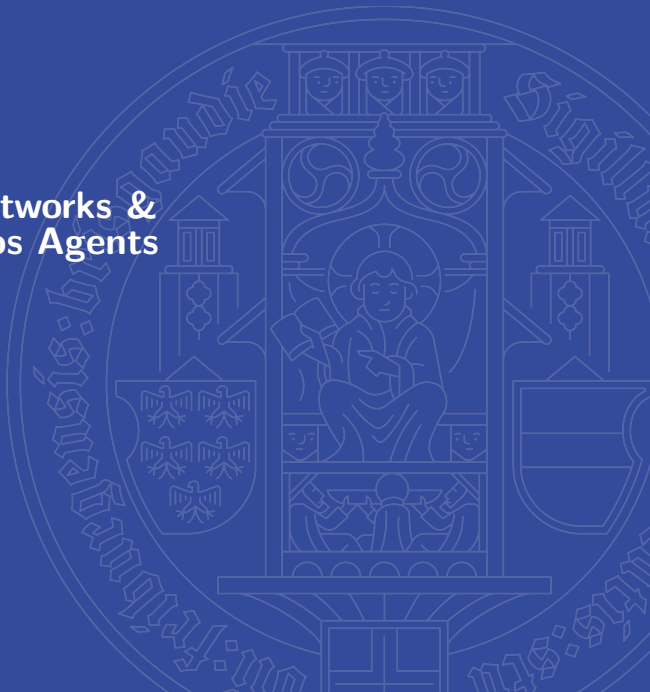
Agent Modeling with Neural Networks & Environments with Homogenouos Agents

Multiagent Reinforcement Learning Seminar

Mahdi Gheidi
Seong Jin You

Neurorobotics Group
Department of Computer Science
University of Freiburg

July 15th 2026



Outline

Why Agent Modeling?

- In **any** multi-agent environment, an agent must consider the actions of other agents to learn an effective policy.
- But so far, this happened only *indirectly*:
  - Independent learning: other agents only enter through the training data they generate.
  - Multi-agent policy gradients / value decomposition: centralized critics *can* be conditioned on others' actions — but only during training.
- Without explicit knowledge of others' policies:
  - the environment appears **non-stationary** — other agents continually learn and change their policies,
  - we cannot select **best responses** to what others will actually do.

Agent modeling: explicitly model the policies of other agents
$\Rightarrow$ predict their actions, adapt to their changing behavior.

Recap: Tabular Agent Modeling (Section 6.3)

Idea (JAL-AM): count how often agent j picked action a_j in state s and use the **empirical action distribution** as a model:

$$\hat{\pi}_j(a_j \mid s) = \frac{C(s, a_j)}{\sum_{a'_j \in A_j} C(s, a'_j)}$$

Act by best response against the models:

$$AV_i(s, a_i) = \sum_{a_{-i} \in A_{-i}} Q_i(s, \langle a_i, a_{-i} \rangle) \prod_{j \neq i} \hat{\pi}_j(a_j \mid s)$$

Limitation

Like tabular value functions, count-based models cannot **generalize** to novel states in which the actions of other agents have not been observed yet.

$\Rightarrow$ Use **deep neural networks** to learn generalizable agent models.

Joint-Action Learning with Deep Agent Models

Agent models

Each agent i maintains agent models $\hat{\pi}^i_{-i} = \{\hat{\pi}^i_j\}_{j \neq i}$ of the policies of all other agents, where $\hat{\pi}^i_j$ is the agent model maintained by agent i for the policy of agent j and is parameterized by ϕ^i_j.

- Each model is a neural network: **input** = observation history h^t_i of agent i, **output** = probability distribution over the actions of agent j.
- Trained during training time from the *observed* actions of other agents, by minimizing the cross-entropy loss:

$$\mathcal{L}(\phi^i_j) = -\log \hat{\pi}^i_j(a^t_j \mid h^t_i; \phi^i_j)$$

= maximizing the likelihood of the observed action a^t_j of agent j given agent i's own observation history.

Training the Joint-Action Value Function

Each agent additionally trains a **centralized action-value function** $Q(h_i, \langle a_i, a_{-i} \rangle; \theta_i)$, DQN-style, by minimizing:

$$\mathcal{L}(\theta_i) = \frac{1}{B} \sum_{(h_i^t, a^t, r_i^t, h_i^{t+1}) \in B} \left(r_i^t + \gamma \max_{a_i' \in A_i} AV(h_i^{t+1}, a_i'; \bar{\theta}_i) - Q(h_i^t, \langle a_i^t, a_{-i}^t \rangle; \theta_i) \right)^2$$

- $\bar{\theta}_i$: parameters of the target network.
- The target uses the **greedy action of agent** i and the action probabilities of all other agents **predicted by their agent models** — how exactly? $\rightarrow$ next slide.

Action Values under the Agent Models

Expected value of action a_i, given what the models predict the others will do:

$$AV(h_i, a_i; \theta_i) = \sum_{a_{-i} \in A_{-i}} Q(h_i, \langle a_i, a_{-i} \rangle; \theta_i)\, \hat{\pi}^i_{-i}(a_{-i} \mid h_i; \phi^i_{-i})$$

$$= \sum_{a_{-i} \in A_{-i}} Q(h_i, \langle a_i, a_{-i} \rangle; \theta_i) \prod_{j \neq i} \hat{\pi}^i_j(a_j \mid h_i; \phi^i_j)$$

- Same structure as tabular JAL-AM — but now Q and $\hat{\pi}^i_j$ are neural networks conditioned on the observation history.
- Problem: the sum runs over **all** joint actions $a_{-i} \in A_{-i}$ — intractable for many agents or large action spaces.

Scaling Up: Sampled Action Values

Instead of enumerating all joint actions, **sample** K joint actions from the agent models and average:

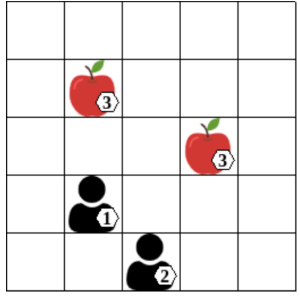
$$AV(h_i, a_i; \theta_i) = \frac{1}{K} \sum_{k=1}^{K} Q(h_i, \langle a_i, a_{-i}^k \rangle; \theta_i), \qquad a_j^k \sim \hat{\pi}_j^i(\cdot \mid h_i)$$

- More efficient, but may introduce additional variance.
- In practice it can even help: sampling concentrates the value estimation on the *most likely* actions of the other agents — exactly where Q has been trained the most.

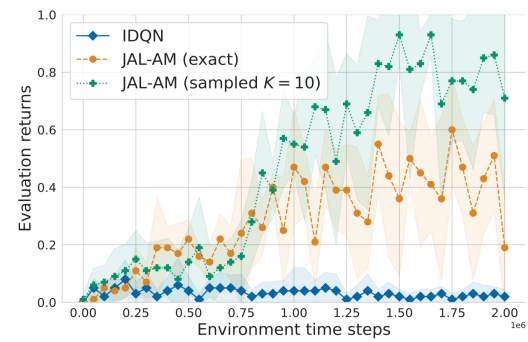
Deep joint-action learning

Initialize n value networks $\theta_1, \ldots, \theta_n$ and target networks $\bar{\theta}_1 = \theta_1, \ldots, \bar{\theta}_n = \theta_n$
Initialize n agent model networks $\phi^1_{-1}, \ldots, \phi^n_{-n}$ and replay buffers $D_1, \ldots, D_n$
for time step $t = 0, 1, 2, \ldots$ **do**
    Collect current observations $o^t_1, \ldots, o^t_n$
    For each agent i: with probability ϵ choose random action a^t_i, else $a^t_i \in \arg\max_{a_i} AV(h_i, a_i; \theta_i)$
    Apply actions $(a^t_1, \ldots, a^t_n)$; collect rewards $r^t_1, \ldots, r^t_n$ and next observations $o^{t+1}_1, \ldots, o^{t+1}_n$
    for agent $i = 1, \ldots, n$ **do**
        Store $(h^t_i, a^t_i, r^t_i, h^{t+1}_i)$ in D_i; sample mini-batch of B transitions $(h^k_i, a^k_i, r^k_i, h^{k+1}_i)$ from D_i
        Targets $y^k_i \leftarrow r^k_i$ if s^{k+1} terminal, else $y^k_i \leftarrow r^k_i + \gamma \max_{a'_i \in A_i} AV(h^{k+1}_i, a'_i; \bar{\theta}_i)$
        Critic loss $\mathcal{L}(\theta_i) \leftarrow \frac{1}{B} \sum_{k=1}^{B} \left(y^k_i - Q(h^k_i, \langle a^k_i, a^k_{-i} \rangle; \theta_i) \right)^2$
        Model loss $\mathcal{L}(\phi^i_{-i}) \leftarrow \sum_{j \neq i} \frac{1}{B} \sum_{k=1}^{B} -\log \hat{\pi}^i_j(a^k_j \mid h^k_i; \phi^i_j)$
        Update θ_i and ϕ^i_{-i} by minimizing $\mathcal{L}(\theta_i)$ and $\mathcal{L}(\phi^i_{-i})$; in a set interval update $\bar{\theta}_i \leftarrow \theta_i$
    end for
end for

JAL-AM in Level-Based Foraging



(a) Environment: 5×5 grid, two agents must *cooperate* to collect any item.



(b) Learning curves of IDQN, JAL-AM (exact), and JAL-AM (sampled, $K = 10$).

IDQN fails to reliably collect items; JAL-AM succeeds. The *sampled* variant learns faster and more reliably than the exact one. Figures taken from [ACS24].

Learning Representations of Agent Policies

Ideally, we would condition our policy and value function directly on the **policies of all other agents**. Why is that not feasible?

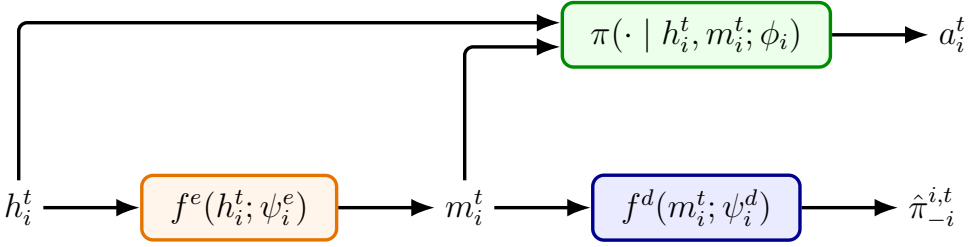
- Other agents' policies may be available during centralized *training*, but not during decentralized *execution*.
- They may be **too complex** to represent explicitly: e.g., the parameters $\phi_{-i} = \{\phi_j\}_{j \neq i}$ of their policy networks are far too many to condition on.
- They **change during learning** — any fixed description becomes stale.

What we want instead

A **compact representation** m_i^t of the other agents' policies that (i) can be inferred from agent i's *own* observations, and (ii) adapts as the other agents' policies change.

$\Rightarrow$ Learn it with an **encoder-decoder** architecture: the encoder f^e maps h_i^t to an embedding m_i^t, the decoder f^d reconstructs the other agents' action probabilities from m_i^t.

Agent Architecture



- During **training**, the decoder f^d reconstructs the action probabilities of all other agents from m_i^t.
- During **execution**, the embedding m_i^t conditions the policy of the agent.

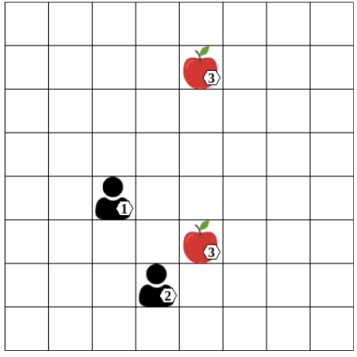
Opponent Model Loss

Encoder and decoder are **jointly** trained to minimize the cross-entropy between predicted action probabilities and the true actions of all other agents:

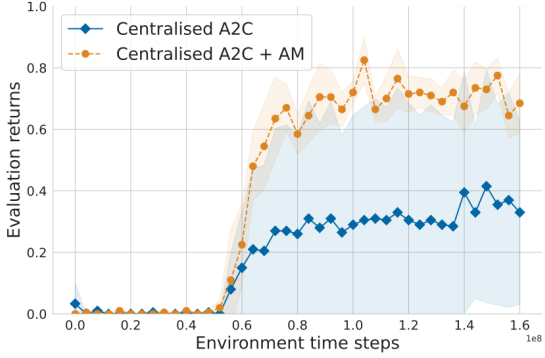
$$\mathcal{L}(\psi_i^e, \psi_i^d) = \sum_{j \neq i} -\log \hat{\pi}_j^{i,t}(a_j^t) \quad \text{with} \quad \hat{\pi}_j^{i,t} = f^d\left(f^e(h_i^t; \psi_i^e); \psi_i^d\right)$$

- The encoder is thereby pushed to store in m_i^t exactly the information that is *indicative of the other agents' action selection*.
- Gradients of the MARL training are **stopped** from flowing into the encoder — it is trained purely by this reconstruction loss.
- **Algorithm-agnostic:** any MARL algorithm can condition its policy $\pi(\cdot \mid h_i, m_i; \phi_i)$ and value function $V(h_i, z, m_i; \theta_i)$ on the learned representations.

Does It Help? Centralized A2C ± Agent Modeling



(a) Environment: 8×8 grid, two agents must cooperate to collect any item.



(b) Centralized A2C with and without representation-based agent models. Figures from [ACS24].

With agent models: faster learning, higher converged returns, and lower variance across runs.

Agent Modeling: Takeaways

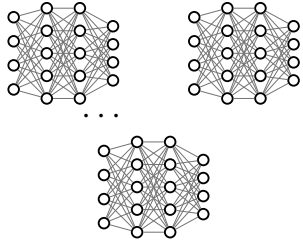
- Agent modeling makes the influence of other agents **explicit** instead of implicit.
- **JAL-AM with deep agent models:** reconstruct other agents' policies with neural networks; select best responses via model-weighted action values; sample K joint actions to scale.
- **Representation learning:** when policies are unavailable, too complex, or changing, learn a compact embedding m_i^t with an encoder-decoder and condition the policy on it — works with *any* MARL algorithm.

Outline

Homogeneous agents: Motivation

Previously, we have seen environments and agents in which we have used independent learning algorithms such as DQN. Problem? The huge number of parameters and the slow training process.

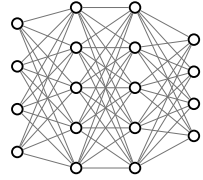
Multiple independent agents



Each of the K agents has its own network

$K \times N$ parameters

Shared network

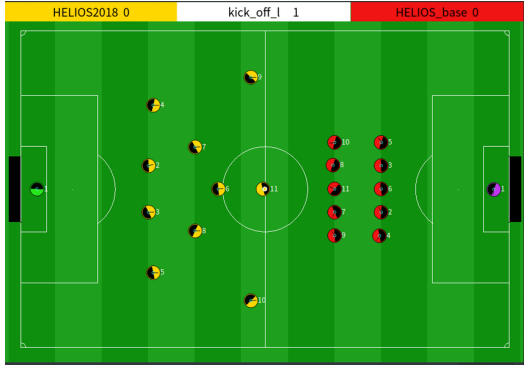


All agents use the same network

N parameters

Homogeneous Agents: Motivation

Starcraft: a game with Agents with different roles, and usually different goals



Robocup Simulation: The agents have the same action space, same goals and observation abilities, and are interchangeable

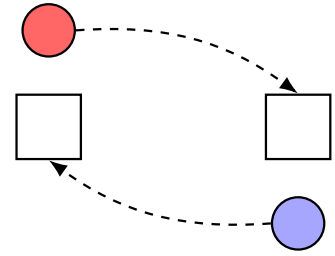
Weakly Homogeneous Agents

Definition: Weakly homogeneous agents

An environment has weakly homogeneous agents if for any joint policy $\pi = (\pi_1, \pi_2, \ldots, \pi_n)$ and permutation between agents $\sigma : I \to I$, the following holds:

$$U_i(\pi) = U_{\sigma(i)}\left(\left\langle \pi_{\sigma(1)}, \pi_{\sigma(2)}, \ldots, \pi_{\sigma(n)} \right\rangle\right), \quad \forall i \in I$$

Intuition: agents can *swap* their policies without changing the expected returns — the agents are interchangeable.



Agents have to move to two *different* locations.

Strongly Homogeneous Agents

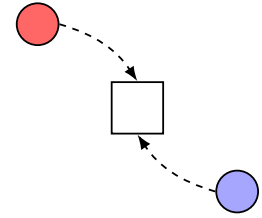
But what if agents are weakly homogeneous and even the goals are the same?

Definition: Strongly homogeneous agents

An environment has strongly homogeneous agents if the optimal joint policy consists of identical individual policies, formally:

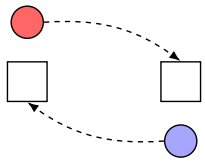
- The environment has weakly homogeneous agents.
- The optimal joint policy $\pi^* = (\pi_1, \pi_2, \ldots, \pi_n)$ consists of identical policies $\pi_1 = \pi_2 = \cdots = \pi_n$

Intuition: not only are the agents interchangeable — at the optimum they all *behave identically*.



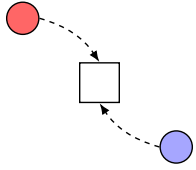
Both agents have to move to the *same* location.

Homogeneous vs. Non-Homogeneous Agents



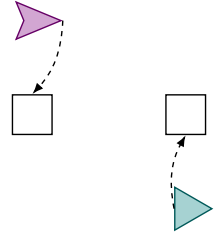
(a) Weakly homogeneous agents

Agents are interchangeable, but must move to different locations.



(b) Strongly homogeneous agents

Both agents can move to the same location.



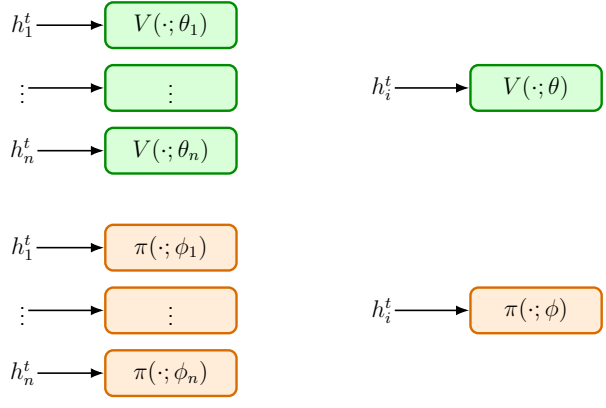
(c) Non-homogeneous agents

Agents are inherently different: different roles, capabilities, or action spaces.

But if we have homogeneity, how can we use this for our gain?

Parameter sharing

When having strongly homogeneous agents, we know that all agents use identical policies. So do we need to keep different parameters for each? Answer is NO



Parameter Sharing: A Free Lunch?

Pros

- Number of parameters stays **constant** in the number of agents ⇒ less compute, faster training
- Shared parameters are updated with the experience of **all** agents ⇒ more diverse data, better sample-efficiency

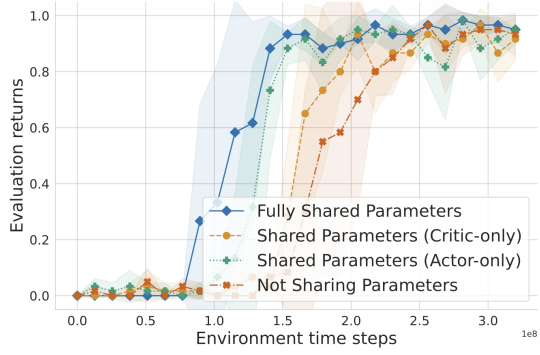
Cons

- Relies on **strongly** homogeneous agents — a strong assumption that is hard to verify, and often simply not true
- Forces **identical** policies: with only weakly homogeneous agents, the optimal joint policy cannot even be represented

⇒ Can we keep the benefits *without* forcing identical policies?

Idea: keep separate parameters, but share the *experience*.

Parameter sharing in LBF, a comparison



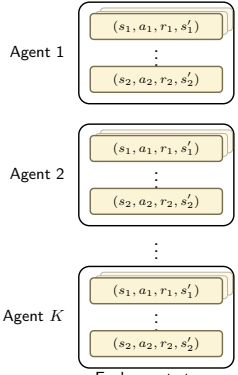
Learning curves for independent A2C with and without parameter sharing on level-based foraging. Figure taken from [ACS24].

Experience sharing

So, how do we exploit homogeneity properties in environments with weakly homogeneous agents?
Agents may need to learn different policies but that doesn't mean that they cannot use experiences of each other.
How?

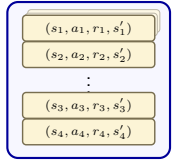
Independent vs. Shared Replay Buffers

Independent replay buffers



Each agent stores transitions in its own replay buffer

Shared replay buffer



All agents store transitions in one shared replay buffer

Caveat of experience sharing

- The shared replay buffer works because DQN is **off-policy**: it can learn from transitions generated by *any* policy — including other agents' policies.
- **On-policy** algorithms such as A2C or PPO require trajectories sampled from the *current* policy being optimized ⇒ we cannot directly feed them other agents' experience.

So, is experience sharing off-limits for on-policy methods?

No — Shared Experience Actor-Critic (SEAC) uses *importance sampling* to correct for transitions collected by other agents.

Recap: Importance Sampling [SB18]

Estimating expectations under a different distribution

We can estimate an expectation under a *target* policy π using samples generated by a *behavior* policy π_β by re-weighting each sample with the **importance sampling ratio**:

$$\mathop{\mathbf{E}}_{a\sim\pi}[f(a)] = \mathop{\mathbf{E}}_{a\sim\pi_\beta}\left[\frac{\pi(a)}{\pi_\beta(a)}\, f(a)\right]$$

Applied to the policy gradient, the loss for off-policy optimization from a behavioral policy π_β becomes:

$$\mathcal{L}(\phi) = -\frac{\pi(a^t|h^t;\phi)}{\pi_\beta(a^t|h^t)}\left(r^t + \gamma V(h^{t+1};\theta) - V(h^t;\theta)\right)\log\pi(a^t \mid h^t;\phi)$$

$\Rightarrow$ Exactly the correction SEAC needs to reuse *other agents'* on-policy trajectories.

SEAC [ACS24]

$$\mathcal{L}(\phi_i) = -\left(r_i^t + \gamma V(h_i^{t+1};\theta_i) - V(h_i^t;\theta_i)\right)\log \pi(a_i^t \mid h_i^t;\phi_i)$$

$$-\lambda \sum_{k\neq i} \underbrace{\frac{\pi(a_k^t \mid h_k^t;\phi_i)}{\pi(a_k^t \mid h_k^t;\phi_k)}}_{\text{importance sampling}} \left(r_k^t + \gamma V(h_k^{t+1};\theta_i) - V(h_k^t;\theta_i)\right)\log \pi(a_k^t \mid h_k^t;\phi_i)$$

- **Own experience:** the standard on-policy actor-critic loss of agent i.
- **Shared experience:** trajectories of all other agents $k \neq i$, treated as off-policy data and corrected via importance sampling — this is what makes experience sharing possible in an *on-policy* setting.

Outline

What did we discuss in Chapter 9?

Section	Technique	Challenge addressed (from Ch. 5)
9.3 Independent Learning	Treat others as environment	Baseline — exposes non-stationarity
9.4 Policy Gradients	Centralized critics	Credit assignment, non-stationarity (training)
9.5 Value Decomposition	VDN, QMIX	Credit assignment, at scale
9.6 Agent Modeling	Deep JAL-AM, representations	Non-stationarity, coordination with diverse agents
9.7 Homogeneous Agents	Parameter & experience sharing	Scaling, sample efficiency
9.8 Self-Play	MCTS, AlphaZero	Equilibrium-finding, 2-player zero-sum
9.9 Population-Based Training	PSRO, AlphaStar	Equilibrium-finding & robustness, general competitive

Reference

[ACS24] Stefano V Albrecht, Filippos Christianos, and Lukas Schäfer.
Multi-agent reinforcement learning: Foundations and modern approaches.
MIT Press, 2024.

[CSA20] Filippos Christianos, Lukas Schäfer, and Stefano Albrecht.
Shared experience actor-critic for multi-agent reinforcement learning.
Advances in neural information processing systems, 33:10707–10717, 2020.

[SB18] Richard S Sutton and Andrew G Barto.
Reinforcement learning: An introduction.
MIT press, 2018.